

AI 101 на GigaCode

Документ для докладчика — Part 1 Updated

Новый PDF, актуализированный по presentation-part1.html. Старый файл не перезаписывается. Дата факт-чека: 2026-05-21. Источник истины по структуре и содержанию: презентация Part 1.

ФОКУС PART 1

- AI, ML, нейросети и LLM.
- LLM vs классический ML.
- Типы моделей и выбор инструмента.
- Интуиция работы нейросети и LLM: параметры, inference, токены, embeddings, attention, decoding.
- Контекст, context management, токены и цена.
- Prompt engineering как мини-спецификация.

ОБЩИЙ ТАЙМИНГ

Ориентировочно 63 минуты, если проговаривать все заметки. Для 45-50 минут: сжать блок 4 до примеров выбора, в блоке 5 не углубляться в dense/MoE и параметры, а блоки 8-9 вести через практические примеры.

ГЛАВНАЯ МЫСЛЬ

LLM не достаёт истину из базы знаний. Она строит вероятностное продолжение на основе текущего контекста. Поэтому инженерное качество определяется сборкой контекста, постановкой задачи и проверкой результата.

Блок 1. Введение и мотивация

ПРИВЯЗКА К ПРЕЗЕНТАЦИИ

Слайды 1-5: title, оглавление, рамка доклада, два контекста, переход

ВРЕМЯ

5 минут

ЦЕЛЬ

Задать рамку Part 1: это практичный AI 101 для инженеров и смежных ролей. Главная мысль первой части: качество ответа зависит не только от модели, а от того, что попало в текущий контекст и как поставлена задача.

КРАТКИЕ ТЕЗИСЫ

- Не разбираем математику трансформеров и backpropagation.
- Не обсуждаем сценарий «AI заменит всех» и маркетинговые обещания.
- Разбираем интуицию LLM, контекст, context management и prompt как мини-спецификацию.
- Формула доклада: модель + контекст + инструменты + workflow + проверка.
- Один и тот же вопрос без контекста даёт правдоподобный общий ответ; с файлами, тикетом и границами задачи — конкретный и проверяемый ответ.

ТЕКСТ ДОКЛАДЧИКА

Начать стоит с ожиданий. Этот доклад не делает слушателя ML-инженером и не обещает магического роста продуктивности. Он даёт инженерную картину: что модель видит, как она строит ответ и почему одинаковая модель может быть полезной или бесполезной в зависимости от входа.

Показать пример refreshToken. Без контекста модель вынуждена отвечать общими словами: проверьте логи, обработку исключений, базу данных. С контекстом — конкретные файлы, тикет, ожидаемый код ответа — появляется возможность связать симптом с реализацией. Это и есть практическая тема Part 1: не «какая модель умнее», а что мы положили в окно модели.

Здесь полезно проговорить, что аудитория будет видеть два уровня одновременно. Первый уровень — как LLM устроена концептуально: контекст, токены, вероятности и ограничения. Второй уровень — как этим пользоваться в работе: не бросать модели расплывчатую просьбу, а давать ей источник истины, границы задачи и критерии проверки. Тогда слушатели сразу понимают, что Part 1 не оторван от практики, а подготавливает фундамент для CLI-workflow в следующих частях.

На примере refreshToken можно отдельно подчеркнуть разницу между «звучит разумно» и «можно проверить». Ответ без контекста не обязательно плохой по форме: он вежливый, связный и похож на совет опытного разработчика. Но у него нет привязки к конкретному коду. Ответ с контекстом ценен тем, что его можно открыть в файле, проверить по строке, сверить с тикетом и превратить в следующий инженерный шаг.

Блок 2. AI, ML, нейросети и LLM

ПРИВЯЗКА К ПРЕЗЕНТАЦИИ

Слайд 6

ВРЕМЯ

5 минут

ЦЕЛЬ

Развести базовые термины и показать, что LLM — заметный, но не единственный класс AI-систем.

КРАТКИЕ ТЕЗИСЫ

- AI — широкий зонтик для систем, выполняющих задачи с признаками интеллектуального поведения.
- ML — подход, где поведение настраивается по данным, а не полностью описывается правилами.
- Нейросети — один класс ML-моделей; это вычислительные системы с параметрами, а не «цифровой мозг».
- LLM — большая языковая модель, работающая с токенами, текстом, кодом и инструкциями.
- LLM не равна всему AI: рекомендации, OCR, скоринг, маршрутизация и прогнозы могут решаться другими методами.

ТЕКСТ ДОКЛАДЧИКА

Здесь важно снять терминологическую кашу. AI часто используют как общий ярлык для всего: от OCR и рекомендаций до чат-ботов и coding agents. В практической работе полезнее думать слоями. Самый широкий слой — AI. Внутри него ML. Внутри ML есть нейросети. LLM — один из заметных классов нейросетевых моделей, потому что язык оказался универсальным интерфейсом к задачам.

Не стоит спорить о строгих академических границах. Для аудитории важнее прикладной вывод: если задача стабильная, табличная и хорошо измеримая, LLM может быть лишним инструментом. Если задача выражается языком, кодом, требованиями, документацией или диалогом, LLM становится естественным кандидатом.

Если аудитория смешанная, лучше не перегружать определениями. Достаточно дать рабочую карту: AI — общий термин, ML — способ учиться на данных, нейросети — один из классов ML, LLM — нейросетевая модель, для которой язык и код являются основным рабочим материалом. Такая карта нужна не ради терминов, а чтобы дальше не возникало ожидания, что LLM обязана быть лучшим ответом на любую AI-задачу.

Можно привести бытовое сравнение: OCR распознаёт текст на изображении, скоринговая модель оценивает вероятность события, рекомендательная система ранжирует варианты, а LLM помогает формулировать, объяснять, анализировать и преобразовывать информацию. Все эти вещи могут называться AI, но инженерные свойства у них разные: разные входы, разные ошибки, разные способы проверки.

Блок 3. LLM vs классический ML

ПРИВЯЗКА К ПРЕЗЕНТАЦИИ

Слайд 7

ВРЕМЯ

5 минут

ЦЕЛЬ

Показать, что LLM и классический ML решают разные классы задач и имеют разные критерии качества.

КРАТКИЕ ТЕЗИСЫ

- Классический ML обычно имеет заранее заданный вход, тип выхода и метрику качества.
- LLM принимает задачу в языковой форме и может возвращать текст, код, план, анализ или вопросы.
- Классический ML чаще выигрывает в стабильных массовых задачах: скоринг, фрод, прогноз нагрузки.
- LLM сильна там, где задача плохо формализована и требует работы с контекстом.
- LLM не заменяет ML; это другой инструмент для задач, где язык и контекст являются интерфейсом.

ТЕКСТ ДОКЛАДЧИКА

В таблице на слайде полезно проговорить не только различие входов и выходов, но и различие режима проверки. У классического ML часто есть метрика заранее: precision, recall, AUC, ошибка прогноза. У LLM-ответа метрика часто появляется через ревью результата: правильно ли поняты требования, есть ли ссылки на код, не смешаны ли факты и гипотезы.

Фраза «когда не LLM» должна прозвучать спокойно. Если нужна дешёвая, воспроизводимая классификация 500K обращений в день, то классический ML или даже правила могут быть практичнее. Если нужно понять legасу-фичу по коду, тикетам и тестам, то языковая модель в CLI-среде лучше соответствует форме задачи.

Когда говорим про классический ML, важно назвать его сильные стороны, а не подавать как устаревший подход. Если задача стабильная, данные размечены, нагрузка большая, а качество измеряется одной понятной метрикой, классический ML часто проще эксплуатировать. Он предсказуемее по стоимости, легче валидируется и лучше подходит для массового конвейера.

LLM выигрывает в другом месте: когда входом является не таблица признаков, а смесь кода, требований, логов, документации и человеческой формулировки. Здесь язык становится интерфейсом, а контекст — рабочей областью модели. Поэтому правильный вопрос не «что сильнее», а «какая форма задачи перед нами».

Блок 4. Типы моделей и выбор инструмента

ПРИВЯЗКА К ПРЕЗЕНТАЦИИ

Слайды 8-9

ВРЕМЯ

7 минут

ЦЕЛЬ

Объяснить, что «модель» — не один универсальный объект. В Part 1 акцент на назначении модели и архитектурном trade-off dense/MoE.

КРАТКИЕ ТЕЗИСЫ

- Chat/Instruct: диалог, инструкции, объяснения, черновики.
- Code: код, геро-контекст, diff, тесты, зависимости, conventions.
- Embedding: текст или код превращается в вектор для semantic search и RAG; такая модель обычно не генерирует ответ пользователю.
- Reasoning: многошаговый анализ, баги, миграции, противоречия; часто дороже и медленнее.
- Dense: вся модель участвует в обработке токена; MoE: активируется часть «экспертов», поэтому общее число параметров и активные параметры — разные вещи.
- Выбор делается под задачу: классификация массовых обращений, поиск похожего тикета и документация legacy-фичи требуют разных инструментов.

ТЕКСТ ДОКЛАДЧИКА

Слайд намеренно не превращается в каталог вендоров. Линия доклада: не «самая умная модель», а подходящая. Для semantic search не нужна генеративная LLM на каждом запросе; нужна embedding-модель и индекс. Для reverse engineering фичи по репозиторию полезнее code-ориентированная модель в CLI-среде, потому что там важны файлы, diff, тесты и tool use.

Dense/MoE объяснить аккуратно. «100B параметров» само по себе не означает, что каждый токен будет стоить как полный прогон 100B dense-модели. У MoE часть параметров активируется маршрутизатором. Но это не бесплатная магия: качество, latency, память, стабильность маршрутизации и поддержка инструментов всё равно важны.

На слайде с типами моделей стоит остановиться на anti-pattern: выбирать модель по общему рейтингу или размеру. Для инженерной задачи это слабый критерий. Нужно смотреть на назначение: умеет ли модель работать с кодом, понимает ли длинный геро-контекст, поддерживает ли tool use, насколько предсказуемы latency и стоимость.

В примерах выбора важно показать экономику решений. Классификация обращений на сотни тысяч запросов в день требует дешёвого и стабильного механизма. Поиск похожего тикета лучше решается embedding-моделью и индексом. А документация legacy-фичи требует code-LLM, потому что нужно пройти по зависимостям, тестам, side effects и не перепутать факты с предположениями.

ФАКТ-ЧЕК И УТОЧНЕНИЯ

- Термин GigaCode используется как целевая среда доклада. Технические тезисы Part 1 не зависят от конкретного бренда CLI-агента.
- Факт-чек добавляет оговорку: embedding-модель для RAG и внутренние token embeddings генеративной LLM — родственные по идее векторные представления, но это не обязательно одно и то же пространство или одна и та же модель.

Блок 5. Как работает нейросеть и LLM

ПРИВЯЗКА К ПРЕЗЕНТАЦИИ

Слайды 10-19

ВРЕМЯ

16 минут

ЦЕЛЬ

Дать техническую интуицию без формул: нейросеть как параметризованная функция; обучение и инференс; путь запроса через tokenization, embeddings, attention и decoding.

КРАТКИЕ ТЕЗИСЫ

- Нейросеть можно представить как $f(x; \theta)$: вход, параметры, выход.
- Обучение настраивает параметры; инференс применяет уже обученные параметры к текущему контексту.
- Модель не обучается на нашем репозитории в момент диалога; если знание нужно, оно должно попасть в контекст.
- Tokenization: текст не равен словам; токены могут быть словами, частями слов, символами, пунктуацией и кусками кода.
- Embeddings: токены или фрагменты получают числовые представления; близость в векторном пространстве помогает работать со смысловой похожестью.
- Attention помогает учитывать связи между токенами в текущем контексте.
- Decoding строит ответ токен за токеном из распределения вероятностей; temperature/top-k/top-p меняют стиль выборки, но не делают ответ истинным.
- Top-k оставляет фиксированное число самых вероятных кандидатов. Top-p оставляет столько кандидатов, сколько нужно для заданной суммарной вероятности.

ТЕКСТ ДОКЛАДЧИКА

Сначала дать простую модель: в обычной программе поведение задаётся кодом, а в ML-модели — архитектурой и параметрами, полученными на обучении. Поэтому модель умеет обобщать, но её поведение менее прозрачно и требует проверки.

Раздел training/inference важен для практики. Когда мы работаем в GigaCode, модель не «запоминает» проект между сессиями сама по себе. Она применяет параметры к текущему окну: системные инструкции, история, файлы, outputs tools, текущий запрос. Отсюда прямой вывод: проектные факты нужно явно принести в контекст или через tools, или через правила, или через RAG/поиск.

Живой пример refreshToken удобно использовать как сквозной. Сначала строка разбивается токенизатором. Потом токены получают внутренние представления. Attention связывает refreshToken, 500, 401 и auth-контекст. Decoding не достаёт готовую истину из базы, а выбирает следующий токен. Даже если следующий токен выглядит правдоподобно, истинность зависит от фактов в контексте и последующей проверки.

После decoding-слайда отдельно проговорить фильтры выборки на примере завтрака. Модель уже посчитала вероятности вариантов: каша, яблоко, бутерброд и так далее. Temperature меняет резкость распределения: при $t = 0.1$ модель чаще выбирает самый вероятный вариант, при $t = 1.2$ хвост вариантов получает больше шансов. Top-k = 3 оставляет ровно три самых вероятных варианта. Top-p = 0.85 набирает кандидатов сверху вниз, пока суммарная вероятность не достигнет 85%. После фильтрации модель всё равно сэмплит из оставшегося набора. Эти настройки управляют разнообразием, но не проверяют истинность ответа.

Когда объясняем нейросеть как функцию, полезно сразу снять антропоморфизм. Модель не думает в человеческом смысле и не держит в голове намерение пользователя. Она применяет обученные параметры к текущему входу. Поэтому маленькое изменение входа — например, добавленный файл, уточнение режима или удалённый шумный лог — может заметно изменить результат.

Про temperature, top-k и top-p важно сказать, что это не три ручки истины. Они управляют тем, насколько широко модель выбирает из уже посчитанных вариантов. Низкая температура делает выбор более острым и стабильным, высокая даёт больше разнообразия. Top-k ограничивает число кандидатов, top-p ограничивает суммарную массу вероятности. Но если в контексте нет правильного факта, никакая настройка sampling не сделает ответ проверенным.

ФАКТ-ЧЕК И УТОЧНЕНИЯ

- Оригинальная работа Transformer описывает архитектуру, основанную на attention-механизмах, без рекуррентных и convolutional слоёв в базовой encoder-decoder схеме.
- Современные Transformer-токенизаторы часто используют subword tokenization: слово, редкий термин или camelCase-идентификатор могут быть разбиты на несколько частей.
- Числа токенов на слайде — иллюстративные. Точное разбиение зависит от конкретного токенизатора модели.

Блок 6. Контекст, память и RAG

ПРИВЯЗКА К ПРЕЗЕНТАЦИИ

Слайды 19-21

ВРЕМЯ

6 минут

ЦЕЛЬ

Показать, что контекст — это всё, что модель видит сейчас, и отделить его от памяти и RAG.

КРАТКИЕ ТЕЗИСЫ

- Контекст — текущая видимая моделью информация: system, project rules, history, files, tool results, user prompt.
- Context window ограничено токенами; большой объём не гарантирует хороший ответ.
- Memory — внешний механизм, который сохраняет факты между сессиями и подставляет их в контекст.
- RAG — поиск релевантных внешних документов перед ответом и добавление найденных фрагментов в контекст.
- Практический вопрос один: что попадёт в окно модели перед ответом?

ТЕКСТ ДОКЛАДЧИКА

Контекст — не только последняя реплика пользователя. В coding agent это особенно заметно: модель видит системные правила, инструкции проекта, историю, результаты shell-команд, найденные файлы, ответы tools и иногда summaries. Если внутри этого набора есть шум или противоречия, модель будет работать с ними как с частью входа.

Память и RAG не являются магической долговременной памятью модели. Они полезны только в той мере, в какой система правильно выбрала, сохранила или нашла факты и положила их в текущий контекст. Поэтому инженерная ответственность остаётся прежней: отобрать релевантное, отделить данные от инструкций и проверить выход.

Здесь стоит проговорить практическую ловушку: люди часто говорят «модель знает проект», хотя на самом деле проектные факты либо сейчас лежат в контексте, либо были найдены инструментом, либо сохранены внешней памятью и снова подставлены в контекст. Для модели в момент ответа существует только текущий вход. Всё остальное — механизмы подготовки этого входа.

RAG лучше объяснять не как отдельную магию, а как disciplined search before answer. Сначала система ищет релевантные документы, затем кладёт найденные фрагменты в окно модели, а потом модель генерирует ответ с опорой на эти фрагменты. Если поиск нашёл не то, или фрагменты устарели, генерация тоже будет уверенно опираться не на те данные.

ФАКТ-ЧЕК И УТОЧНЕНИЯ

- Исследования long context показывают: модель может использовать информацию неравномерно; релевантные факты в середине длинного контекста часто извлекаются хуже, чем факты в начале или конце.

Блок 7. Токены, стоимость и latency

ПРИВЯЗКА К ПРЕЗЕНТАЦИИ

Слайд 22

ВРЕМЯ

4 минуты

ЦЕЛЬ

Связать токены с практическими ограничениями: стоимостью, лимитом контекста и скоростью ответа.

КРАТКИЕ ТЕЗИСЫ

- Платим и ждём не за символы и не за слова, а за входные и выходные токены.
- Русский текст, JSON, логи и stack traces могут занимать больше токенов, чем ожидается визуально.
- 128K context означает токены, а не символы; часть окна уйдёт на system/project/history/tool outputs.
- Раздутый контекст дороже, медленнее и часто хуже для качества ответа.
- Инженерный вывод: длинные логи и JSON лучше сжимать до summary, если точные строки не нужны.

ТЕКСТ ДОКЛАДЧИКА

Этот слайд нужен как экономический аргумент за context management. Даже если модель технически принимает длинный контекст, это не значит, что нужно грузить всё подряд. Лишние токены увеличивают стоимость, задержку и вероятность того, что важные факты потеряются среди шума.

Нужно явно сказать, что пропорции на слайде ориентировочные. У разных моделей разные токенизаторы. Но направление верное: структурированные дампы, stack traces и длинные машинные форматы быстро съедают окно.

Этот блок можно связать с ежедневной инженерной практикой: длинный stack trace удобно целиком бросить в модель, но часто достаточно верхнего исключения, нескольких relevant frames и краткого описания окружения. Так мы уменьшаем стоимость и latency, а ещё помогаем модели не потерять главный сигнал.

Полезная формулировка для аудитории: токены — это бюджет внимания и денег одновременно. Мы платим за них, ждём из-за них и конкурируем ими внутри context window. Поэтому чистый контекст — это не эстетика, а инженерное ограничение, похожее на память, CPU или размер payload.

Блок 8. Context management: проблемы и решение

ПРИВЯЗКА К ПРЕЗЕНТАЦИИ

Слайды 23-24

ВРЕМЯ

9 минут

ЦЕЛЬ

Разобрать типовые причины плохих ответов и дать практические правила сборки контекста.

КРАТКИЕ ТЕЗИСЫ

- Проблемы: шум, переполнение, устаревшие данные, конфликт инструкций, prompt injection, context drift.
- Шум: важные файлы теряются среди старых docs, lock-файлов, нерелевантных тестов и длинных логов.
- Устаревший контекст: README, ADR или тикет могут описывать прошлую версию системы.
- Prompt injection: внешний документ может содержать инструкции, которые конкурируют с правилами модели.
- Context drift: длинная сессия накапливает отменённые гипотезы и лишние outputs.
- Решение: сузить scope, проверить источники, разделить роли инструкций/данных/формата, сжать шум, отделить внешние данные от инструкций, задать режим CLI-агента.

ТЕКСТ ДОКЛАДЧИКА

Этот блок должен звучать как инженерная практика, а не как теория. Когда модель ошиблась, причина не всегда в слабой модели. Очень часто она получила плохой вход: устаревший документ, противоречивую инструкцию, слишком много логов или задачу без границ.

Хороший контекст — не максимальный. Он минимальный, актуальный, непротиворечивый и привязанный к задаче. Для reverse engineering auth-фичи не нужен весь репозиторий; нужны релевантные файлы, тесты, текущий симптом, вторичные источники вроде тикета и явный режим CLI-агента: только анализ, только план, правки после подтверждения или review-only.

Отдельно подчеркнуть безопасность. Jira, wiki, письма, HTML, PDF и найденные страницы — это данные, а не команды для модели. Если в документе написано «ignore previous instructions», это должно анализироваться как вредоносная или нерелевантная строка, а не выполняться.

Для каждой проблемы можно дать короткий рабочий симптом. Шум — модель отвечает общо и не цепляется за нужный файл. Устаревший контекст — модель уверенно предлагает старую команду или старую архитектуру. Конфликт инструкций — ответ становится компромиссом между несовместимыми правилами. Context drift — агент постепенно делает уже не ту задачу, с которой начинали.

Решение хорошо подавать как чек перед запуском агента: какая конкретная задача, какие источники первичные, какие вторичные, что можно игнорировать, какой режим работы выбран, где стоп-условие. Это превращает context management из абстрактного совета в повторяемый рабочий процесс.

ФАКТ-ЧЕК И УТОЧНЕНИЯ

- OWASP LLM01:2025 выделяет direct и indirect prompt injection; RAG и fine-tuning не устраняют этот риск полностью.
- Практика long-context prompting подтверждает ценность структуры: документы и метаданные нужно отделять разметкой, а запрос/инструкции располагать так, чтобы модель ясно понимала задачу.

Блок 9. Prompt как мини-спецификация

ПРИВЯЗКА К ПРЕЗЕНТАЦИИ

Слайды 25-28

ВРЕМЯ

8 минут

ЦЕЛЬ

Дать прикладной шаблон постановки задач модели без магических фраз.

КРАТКИЕ ТЕЗИСЫ

- Плохой запрос смешивает цель, источник, режим и формат; модель додумывает недостающее сама.
- Хороший prompt задаёт цель, объект, входные данные, режим, формат вывода и стоп-условие.
- Для анализа кода полезен режим `analysis-only` или `review-only`, чтобы агент не начал править файлы раньше времени.
- Формат вывода должен быть проверяемым: `current flow, findings by severity, evidence, open questions`.
- Перед отправкой проверить: цель, объект, режим, формат, стоп-условие.
- Для длинных задач добавлять `recap/compression: confirmed facts, decisions, constraints, open questions, next step`.

ТЕКСТ ДОКЛАДЧИКА

Prompt engineering здесь не набор заклинаний. Это мини-спецификация. Чем меньше модель должна додумывать про цель, источник истины, режим и формат, тем проще ревьюить результат.

Пример с `refresh-token` хорошо показывает разницу. «Напиши документацию» даёт связный текст, где подтверждённые факты и домыслы смешиваются. Структурированный запрос говорит: цель — обратный анализ фичи; объект — `auth/token`; код первичен, тикет вторичен; режим — `analysis-only`; формат — `current flow, findings, questions`; стоп-условие — не писать черновик до подтверждения.

Дополнение к слайдам: в длинных задачах нужно периодически чистить `state`. Попросить модель сжать сессию не в красивое резюме, а в рабочее состояние: что подтверждено, что отменено, какие ограничения актуальны, какие вопросы открыты, какой следующий шаг. Это снижает влияние `context drift`.

Важно не создать у аудитории впечатление, что хороший prompt обязан быть длинным. Хороший prompt обязан быть однозначным. Иногда это пять строк: цель, объект, источник истины, формат вывода и стоп-условие. Если эти элементы есть, модель меньше импровизирует там, где нужна инженерная дисциплина.

В конце блока можно дать практическую привычку: перед отправкой запроса спросить себя, сможет ли другой инженер по этому же prompt понять, что делать и как проверить результат. Если нет, модель тоже будет додумывать. Prompt как `mini-spec` полезен именно тем, что делает результат сравнимым, ревьюируемым и пригодным для следующего шага.

ФАКТ-ЧЕК И УТОЧНЕНИЯ

- Для длинных документов официальные `prompt-guides` рекомендуют структурировать документы и метаданные, а также располагать запрос так, чтобы он был явно видим модели.
- Разделение инструкций и данных снижает риск смешения ролей, но не является полной защитой от `prompt injection`; критичные действия требуют внешних ограничений и проверок.

Итог Part 1

- LLM строит вероятностное продолжение, а не гарантированно достаёт истину.
- Качество ответа определяется качеством контекста: минимального, актуального, непротиворечивого и привязанного к задаче.
- Prompt — это мини-спецификация: цель, источник, режим, формат и стоп-условие.
- Ограничения лежат в процессе: context management, tools, workflow и review важнее надежды на «идеальный prompt».

ЧТО ИЗМЕНЕНО ОТНОСИТЕЛЬНО СТАРОГО PDF

- Документ сокращён до Part 1 и выровнен по структуре presentation-part1.html.
- Нумерация блоков соответствует Part 1: контекст — блок 6, токены и цена — блок 7, context management — блок 8, prompt — блок 9.
- Добавлен сквозной пример refreshToken из презентации.
- Добавлены факт-чек оговорки по токенизации, embeddings/RAG, long context и prompt injection.
- Разделы про tools, MCP, GigaCode workflow, skills, sub-agents, hooks и риски следующих частей удалены из основного тайминга Part 1.

Источники факт-чека

Источники использованы для проверки технических формулировок и актуализации оговорок. Основная структура и тезисы документа взяты из presentation-part1.html.

- Attention Is All You Need: <https://arxiv.org/abs/1706.03762>
- Hugging Face Transformers: Summary of tokenizers:
https://huggingface.co/docs/transformers/v4.52.2/tokenizer_summary
- Lost in the Middle: How Language Models Use Long Contexts: <https://arxiv.org/abs/2307.03172>
- Anthropic / Claude docs: long context prompting best practices:
<https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices#long-context-prompting>
- OWASP GenAI Security Project: LLM01:2025 Prompt Injection:
<https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- Model Context Protocol documentation: <https://modelcontextprotocol.io/docs>